Controllers and Routing

ASP.NET Core Web API — Week 2 lecture notes

1. Controllers

A controller is a class that groups related endpoints. It inherits from **ControllerBase** and is marked with **[ApiController]**, which turns on automatic model validation (invalid input returns 400 for you).

2. Attribute routing

```
[ApiController]
[Route("api/courses")]
public class CoursesController : ControllerBase
{
    [HttpGet]                    // GET  /api/courses
    public IActionResult GetAll() { ... }

    [HttpGet("{id:int}")]        // GET  /api/courses/5
    public IActionResult GetById(int id) { ... }

    [HttpPost]                   // POST /api/courses
    public IActionResult Create(CourseRequestDto dto) { ... }
}
```

The **{id:int}** part is a route constraint: /api/courses/abc will not match, so the action never receives bad input.

3. Where does the data come from?

Source	Attribute	Example
URL segment	[FromRoute]	/api/courses/5
Query string	[FromQuery]	/api/courses?search=react
Request body (JSON)	[FromBody]	POST with a JSON object
Form / file upload	[FromForm]	multipart/form-data

4. Returning results

```
return Ok(course);                    // 200 + body
return CreatedAtAction(nameof(GetById),
       new { id = course.Id }, course);  // 201
return NoContent();                   // 204
return NotFound();                    // 404
return BadRequest("Price must be > 0"); // 400
```

5. Keep controllers thin

A controller should only translate HTTP into a method call and back. Rules such as "a course must be open for enrollment" belong in a service, so they can be reused and tested.

Practice

• Add GET /api/books and GET /api/books/{id}.

- Return 404 when a book does not exist.
- Add POST /api/books that returns 201 Created.